

Time and Synchronization



COMS4113

COMS4113: Fundamentals of Distributed Systems

Instructor: Hubertus Franke

Columbia University

<https://frankeh.github.io/cu-ds1-2026-fall>

- Last time: **RPC** — and how **failures** creep into semantics and challenge coordination.
- Today: **time** — and how **unbounded network delays** (*network asynchrony*) creep into semantics and challenge coordination.
- The shape of the argument is the same. A mechanism that looks local and exact (a procedure call; a clock reading) turns out to be neither once it crosses a network.

Two questions today: can we agree on *what time it is*? And when we cannot — can we still agree on *what happened first*?

Physical clocks → NTP, PTP, datacenter time → Lamport clocks → mutual exclusion

- **Physical clocks**
 - Why time is hard
 - Synchronization protocols
 - NTP, PTP, and datacenter time
- Logical clocks
 - Lamport clock protocol
- Examples
 - Global log for debugging
 - Mutual exclusion

Why Is Time Important?

- Needed for **synchronization** and **coordination**.
- Examples:
 - **Mutual exclusion** — who holds the lock, and who asked first?
 - **Barriers** — has every process reached the same point?
 - **Ordering** — which write wins? which event caused which?
- Running example for today: **distributed debugging from logs**.
 - Small enough to do by hand, and it fails in exactly the way real systems fail.

Example: Distributed Debugging

M1 (front end)

```
...  
recv from cli  
...  
send to M2  
...  
recv from M2  
...  
send to cli  
...
```

M2 (app server)

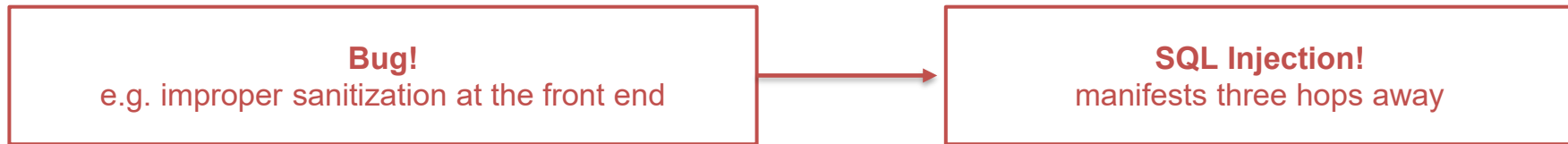
```
...  
recv from M1  
...  
send to M3  
...  
recv from M3  
...  
send to M1  
...
```

M3 (DB server)

```
...  
recv from M2  
...  
SQL query  
...  
send to M2  
...  
...  
...
```

Each machine keeps its **own** log. No machine sees the whole request.

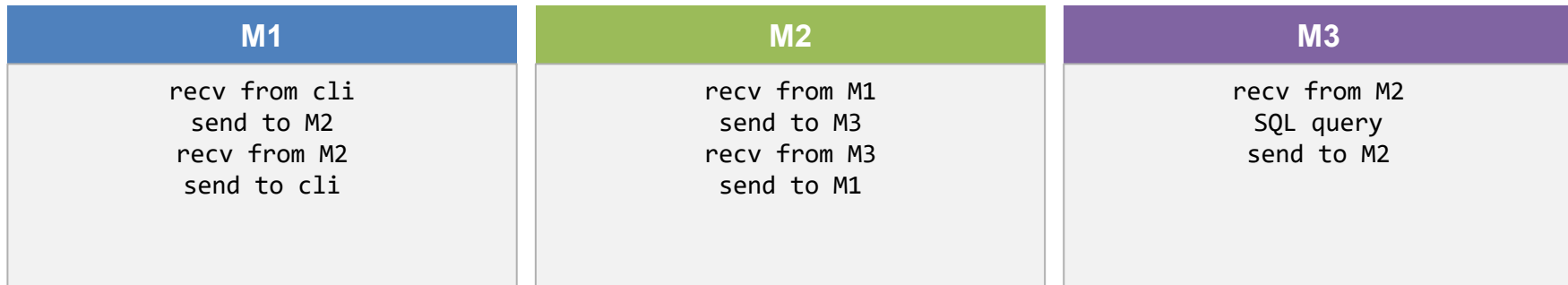
Example: Distributed Debugging



```
SELECT * FROM Users WHERE id='123'; DELETE * FROM Users
```

The cause and the symptom are on different machines.

What We Want: One Global Log



Global log (admin)

M1: recv from cli → M1: send to M2 → M2: recv from M1
M2: send to M3 → M3: recv from M2 → M3: SQL query
M3: send to M2 → M2: recv from M3 → M2: send to M1
M1: recv from M2 → M1: send to cli

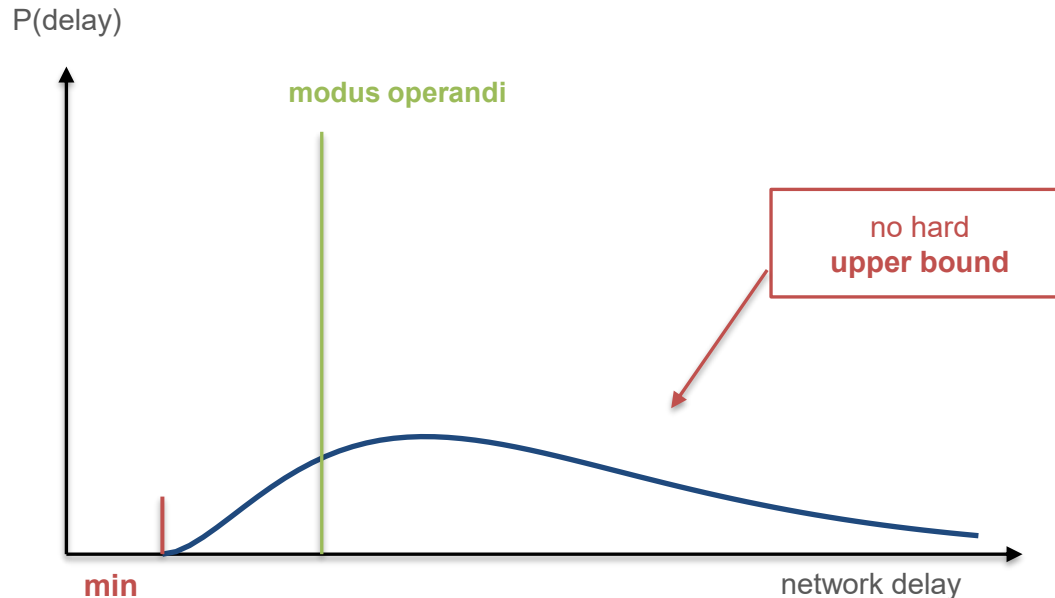
Question: how do we build this? **First answer:** timestamp with the physical clock?

Clock Synchronization Is Hard

- Machines have **different physical clocks**, never structurally identical.
 - Quartz: the crystals themselves differ; ambient temperature and electromagnetic fields shift the oscillator.
 - 1ppm [parts per million]
defined as clock can gain or lose 1usec/sec $\rightarrow$ 86.4msec/day
 - Typical commodity spec: **± 200 ppm (parts per million)** — the same figure TrueTime(*) *and* Sundial(*) assume as worst-case drift. That is ≈ 17 s/day.
- Correcting the resulting offset **involves the network** — whose delays vary over time and, in general, **cannot be upper bounded**.
 - So the fix for clock error is itself subject to an error you cannot bound. That is the whole problem in one sentence.

(*): used by Google Spanner (globally distributed DB) for time synchronization

Asynchronous Networks



- Real networks are **asynchronous**:
 - a lower bound, **min**
 - a typical case, the *modus operandi*
 - **no hard upper bound**
- Assuming a known **max** — the *synchronous* model — is convenient but unrealistic: a buggy router, deep queueing, or an attack breaks it.

- It is not only the **network** that behaves this way.
- **Computation** does too: no operation has a guaranteed execution time — it depends on load, scheduling, GC, page faults, thermal throttling.
- An **asynchronous system** models *both* the network and the computation as having no upper bound.
 - Remember this term. We return to it repeatedly — it is what makes consensus hard, and it is the setting for the FLP (*) result.
- Concretely for clocks: even a *perfectly* synchronized clock read at a known instant can be **stale by the time you act on it**.

(*) Fisher/Lynch/Paterson:

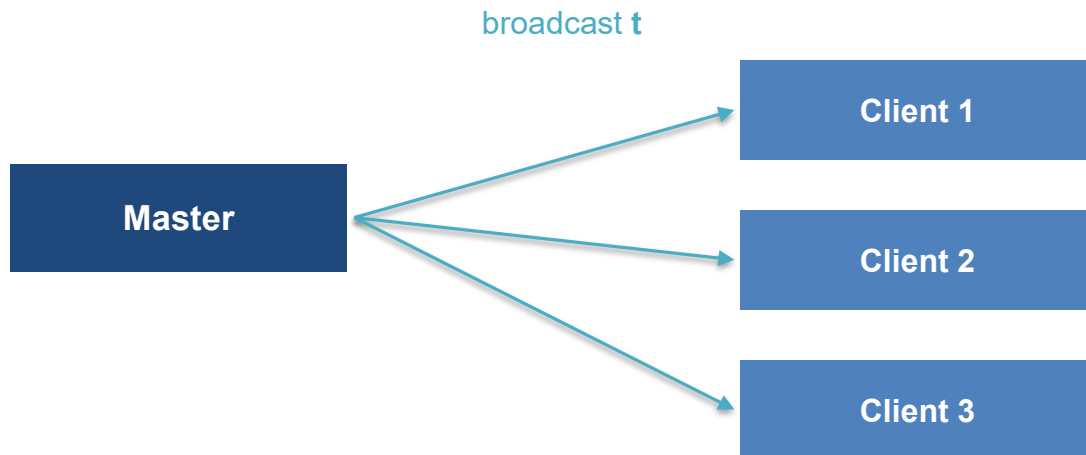
An **asynchronous system** features no upper bounds on network delivery times or processing speeds. The **FLP result** is the foundational mathematical proof that reaches a stark conclusion in this exact setting: it is completely **impossible** for a deterministic consensus protocol to guarantee both agreement (safety) and termination (liveness) if even a single node can unexpectedly crash.

- Best-known algorithm: **NTP**, the Network Time Protocol [Mills-1991]; currently **NTPv4, RFC 5905** (2010).
- Synchronizes to **reference clocks** — GNSS, atomic, or radio — arranged in a hierarchy of *strata*.
- Accuracy, roughly:
 - **WAN: tens of ms** — fine for human coordination, awkward for machine coordination
 - **LAN: sub-ms**; RFC 5905 claims tens of μs as a *potential* on fast LANs
- We first build the **basis** of the protocol from first principles, then look at what a real implementation adds.



The master's clock is **assumed correct** — that assumption is doing real work, and NTP spends most of its complexity trying to earn it (stratum, selection, falsetickers).

Protocol 1: Broadcast-Based Sync

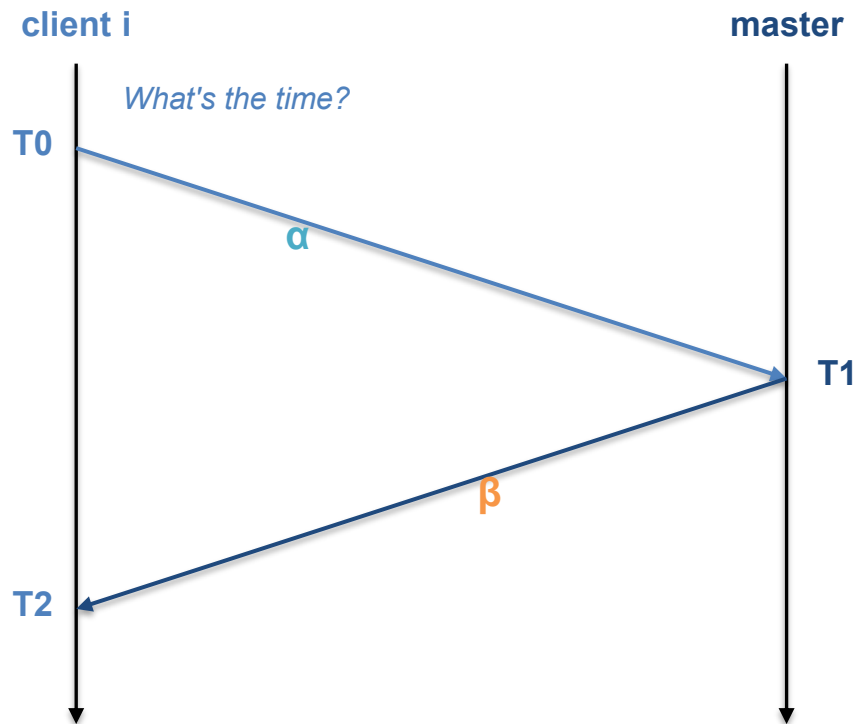


- Master **broadcasts** t to all clients.
- Client i receives at local time t' and sets $C_i(t') := t + \min$
- If a **max** delay existed, the client-master error would be bounded by $(\max - \min)$ — **provably optimal**.
- But $\max$ does not exist. This is the *right* answer to the *wrong* model.

Why it is weak: the client applies a **fixed** correction ($\min$) to a delay that is actually **variable**. Every millisecond of queueing becomes a millisecond of clock error, silently.

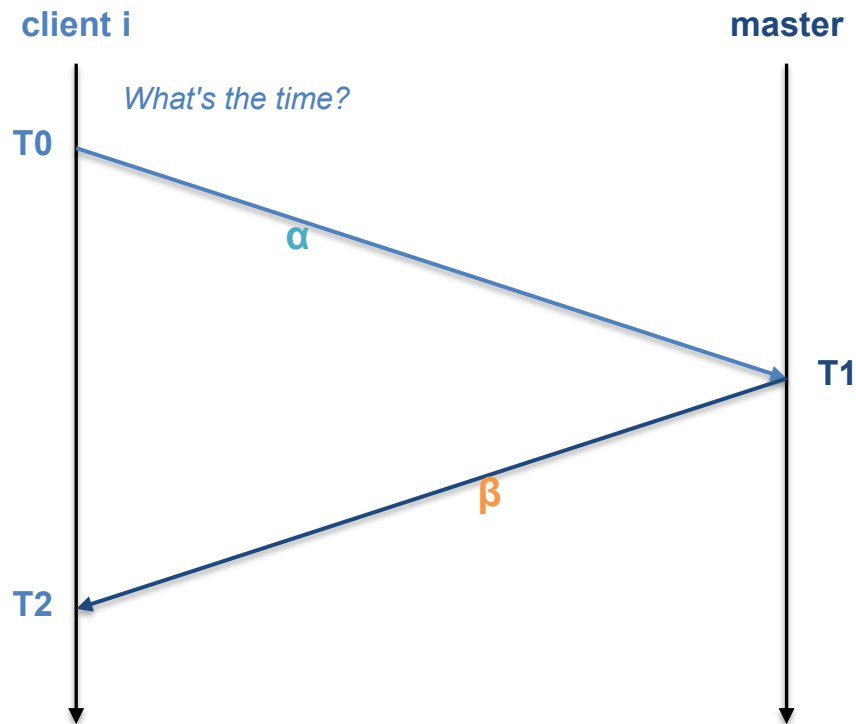
$\min$: empirically measured over time based on RTT (round trip time) between two servers

Protocol 2: Interrogation-Based Sync



- Client i sends a query and saves its local time T_0 .
- Master receives it, takes its local time T_1 , replies with it.
- Client receives the reply at its local time T_2 , and updates its clock.
- **To what value**, to make the error as small as possible?
- Only T_0 , T_1 , T_2 are observable. α and β separately are **not** — and that single fact drives everything that follows.

Protocol 2: The Two Extreme Cases

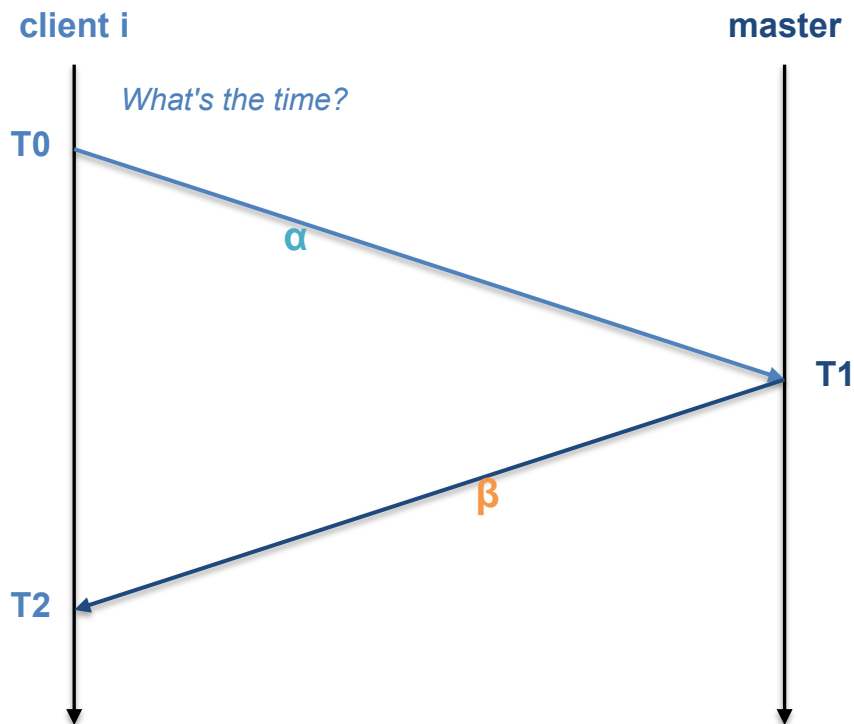


Extreme Case 1: $\alpha = 0$
the request arrives instantly, so
 $\beta = (T_2 - T_0)$
 $C_{\text{master}}(T_2) = T_1 + (T_2 - T_0)$

Extreme Case 2: $\beta = 0$
the reply arrives instantly, so
 $\alpha = (T_2 - T_0)$
 $C_{\text{master}}(T_2) = T_1$

Indistinguishable: same T_0 , T_1 , T_2 either way.
The client knows only the sum $\alpha + \beta$.

Protocol 2: Midpoint and Max Error



Split the difference:
 $C_i(T_2) := T_1 + (T_2 - T_0)/2$

max error = $(T_2 - T_0)/2$
attained in exactly Cases 1 and 2 —
client either behind or ahead

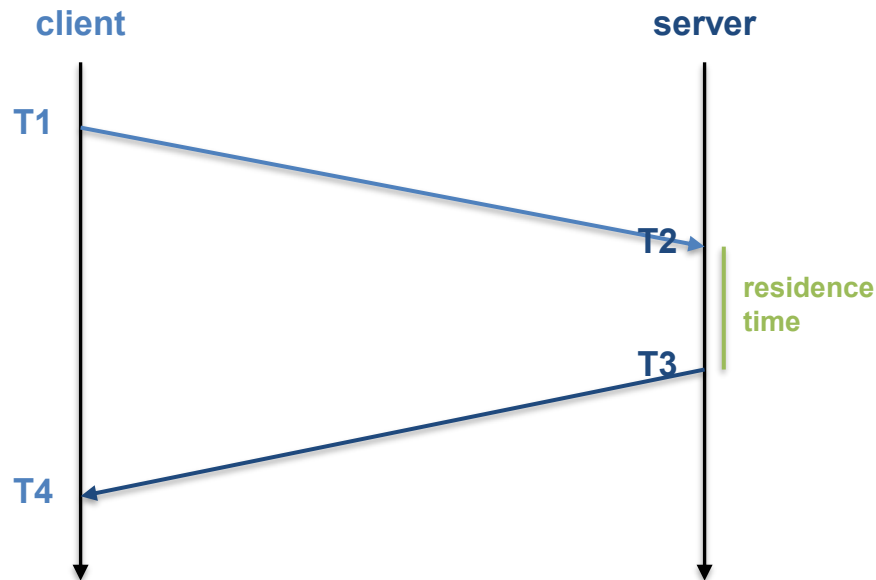
Half the round trip is the price of not knowing
which direction was slow.

What This Analysis Ignores

- **min** — we assumed 0. Real $\text{min} > 0$ shrinks the uncertainty interval.
- **Client drift during $T_0 \dots T_2$** — the clock we are correcting is moving while we measure it.
- **Local delays** — T_2 is taken at one instant but the clock is actually updated later; the gap is unmodelled error.
- **Server residence time** — the master is not instantaneous either.
- **Scalability and fault tolerance** — one master is a bottleneck *and* a single point of failure. Worse: a **lying** master is undetectable to a client with one source.
- Every one of these is something a real protocol must answer. **Next: how NTP answers each.**

- Error shrinks as the measured **RTT approaches $2 \cdot \min$** — a *probabilistic* tradeoff, not a guarantee:
 - **be picky** — accept only near-min samples: tight error bounds, but many trials needed
 - **be sloppy** — accept anything: few trials, loose bounds
- NTP's *clock filter* is exactly this choice, made concrete: keep 8 samples, **use the one with the lowest delay**. A minimum selector, not an average.
- **Network delay bounds clock synchronization, permanently**. No amount of protocol cleverness removes the term.
- GNSS (Global Navigation Satellite System) sidesteps the network path (tiny $\min$, α , β) — but is still non-zero, and introduces an antenna, a cable, and a spoofing surface.

NTP: Four Timestamps, Not Three

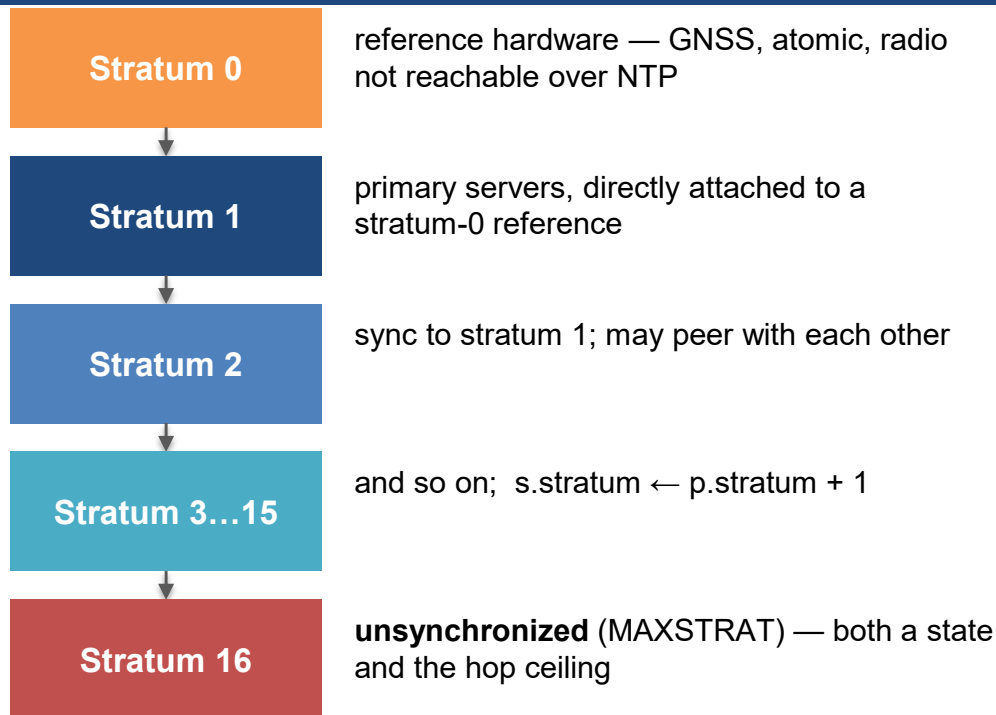


offset	$\theta = \frac{1}{2} * [(T2 - T1) + (T3 - T4)]$
delay	$\delta = (T4 - T1) - (T3 - T2)$

- RFC 5905 names them org, rec, xmt, dst. **T4 is not a wire field** — the client fills it in on arrival.
- Our midpoint formula is this with **residence time collapsed to zero**. NTP subtracts $(T3 - T2)$ because the server is not instantaneous.
- Both still assume **path symmetry**. Asymmetry Δ gives an offset error of $\Delta/2$ — unfixable by measurement.

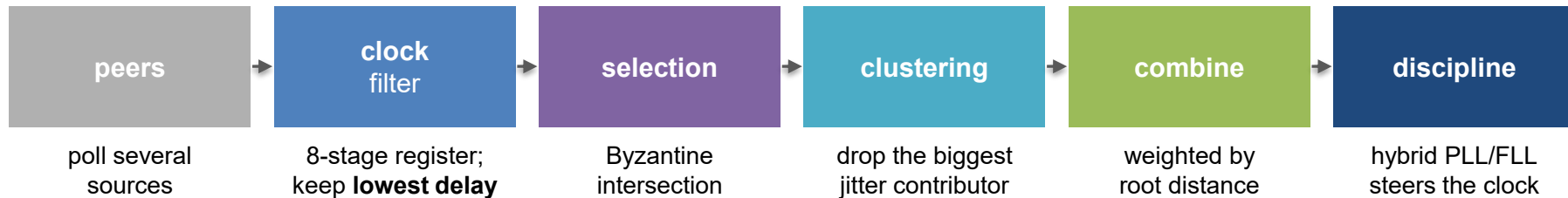
δ can go **negative** (RFC works a case at -6.4 ms), so it is clamped to the clock's precision.

NTP: Stratum Hierarchy



- A received stratum of **0 maps to 16 internally**, so reference clocks pass through the *same* selection and clustering algorithms as anyone else.
- 16 means both *unsynchronized* and *too far away* — the same value carries both meanings.
- 17–255 are reserved.
- Stratum is a **hop count, not a quality score**. A stratum-3 server on a quiet LAN can easily beat a stratum-2 across a congested WAN.

NTP: The Mitigation Pipeline



- **Clock filter** — a *minimum* selector, not an average: the least-delayed sample suffered the least asymmetry.
- **Selection** — Byzantine agreement after Marzullo & Owicki (PODC 1983), "modified to improve accuracy." Tolerates **strictly fewer than half** *falsetickers*, and on failure **refuses to set the clock at all**.
- **Discipline** —
 - a PLL (Phase Locked Loop) when network jitter dominates,
 - a FLL (Freq Locked Loop) when oscillator wander dominates; crossover at the Allan intercept ($ALLAN = 1500 \text{ s}$).

- **Slew** below **128 ms**: nudge the clock's *rate* via `adjtime()`.
 - Ceiling is **0.5 ms/s** (`MAXFREQ 500e-6`) — so "each second of adjustment requires an amortization interval of 2000 s." Correcting 600 s by slewing takes **~14 days**.
- **Step** above 128 ms: jump the clock with `settimeofday()`.
 - A step is **not free**: "all peer data have been invalidated, all associations **MUST** be reset." And it makes time go **backwards** — which breaks code that assumes it cannot.
- **Panic threshold 1000 s**: `ntpd` exits rather than proceed. `-g` waives it once, at startup only.

- **NTS — Network Time Security : RFC 8915 (Sept 2020).** Key establishment over **TLS 1.3 / TCP 4460**, then AEAD-authenticated NTP over UDP 123.
 - Avoid Man-in-the-Middle attacks, where packets are intercepted and wrong time is fed.
 - Uses **opaque, server-stateless cookies** — no per-client state, which is what makes it viable at pool scale.
 - Does **not** encrypt timestamps, and **cannot** stop a symmetric delay attack — an on-path attacker who delays both directions equally is invisible to the math.
- **The bootstrap paradox.** TLS certificate validation needs the correct time; getting the correct time needs TLS. *You must roughly know the time to learn the time.*

- **NTPv5** — draft-ietf-ntp-ntp5-09 (1 July 2026), Lichvar & Mizrahi. **Still a draft, no RFC number.** Specifies only the wire protocol.
 - **Era number on the wire:** the 136-year rollover becomes ~35,000 years
 - **Timescale field** — UTC / TAI / UT1 / leap-smeared UTC, so a smeared server can say so instead of looking like a liar
 - Root-delay resolution $\sim 15 \mu\text{s}$ $\rightarrow$ **$\sim 4 \text{ ns}$** ; clients leak nothing about their own clock; symmetric/broadcast/control modes deleted
- **2036 $\neq$ 2038.** NTP: *unsigned* 32-bit seconds from **1900**, wraps **07 Feb 2036**. Unix: *signed* `time_t` from 1970, overflows 19 Jan 2038.
- **The reference implementation is now the default almost nowhere:** chrony on RHEL/Fedora, systemd-timesyncd on Debian/Ubuntu.
 - 2017 Cure53/CII audit: 14 issues in reference NTP, 8 in NTPsec, chrony rated *trustworthy*. NTPsec deleted $\sim 75\%$ of the code for auditability.

Tighter Than NTP: PTP / IEEE 1588

- PTP: Precision Time Protocol
- Same *math* as NTP — **the same $\frac{1}{2}$, the same symmetry assumption**. PTP wins on **timestamp quality** and **on-path support**, not on cleverer algebra.
- **Hardware timestamping** at the MAC/PHY removes OS and network-stack jitter — the dominant error term for software NTP.
- **Switches participate** instead of adding opaque queueing:
 - *transparent clock* — writes its own residence time into the correction field, compensating queueing without terminating the protocol
 - *boundary clock* — terminates and regenerates PTP
- Hierarchy is **elected** by the BMCA [Best Master Clock Algorithm]
(priority1 → clockClass → clockAccuracy → offsetScaledLogVariance → priority2 → clockIdentity), not configured client-side.
- **Sub-microsecond on a LAN** is the design point.

Comparison NTP vs PTP

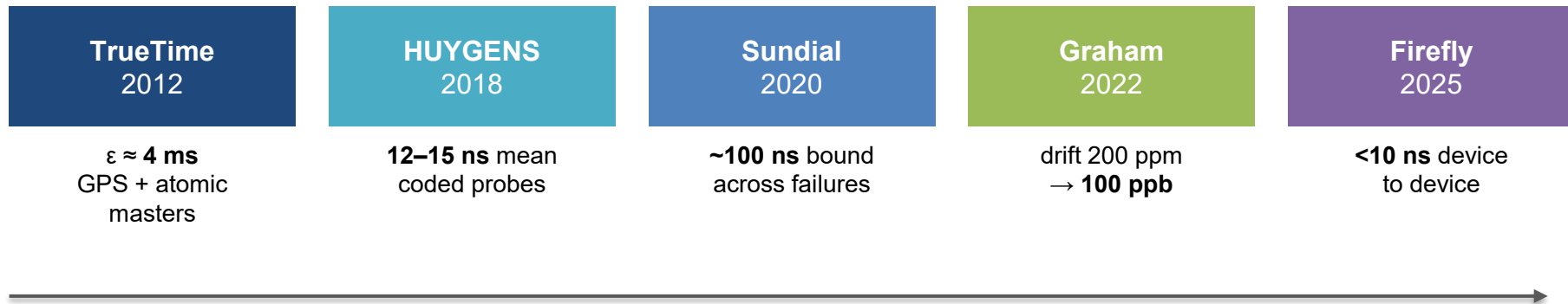
Metric	NTP (stand. servers)	PTP (IEEE 1588)
Typical Precision	1-10+ millisec	10 – 100+ nanosecs
Timestamp Method	Software Stack / Application Layer	Physical layer / Network Hardware
Network Path	Traverses standard unmanaged routers and switches	Requires PTP-aware infrastructure components

- E.g. most popular NICs like Nvidia CX-5/6/7 or Broadcom are PTP compliant

Who Needs Microseconds?

- **Regulation.** MiFID II **RTS 25** (Commission Delegated Regulation (EU) 2017/574) mandates maximum divergence from UTC by trading activity:
 - **100 μ s** for high-frequency trading; **1 μ s** timestamp granularity for the fastest venues
 - A clock is a *compliance artifact*: you must be able to **prove** what your divergence was, which means logging an **error bound**, not just a timestamp.
- **Databases.** Tighter clocks buy throughput directly — Sundial cut Spanner's commit-wait median from **211 μ s** → **49 μ s** (99th: 784 → 238 μ s).
- **Distributed tracing and debugging.** Our running example, at scale: if clock error exceeds inter-service latency, your traces show **effects before causes**.
- **Cloud financial exchanges** now drive sub-10 ns research — the motivation Firefly (SIGCOMM 2025) states outright. Firefly is a USC/Nvidia/Google time synchronization system.
 - Internal Clock Sync : neighbors converge on an average clock value for rapid local synchronization
 - External Clock Sync : converge the internal Clock towards UTC

Datacenter Time: ms to ns (Example Sys)



- **Three different levers.** DTP (SIGCOMM 2016) changes the *physical layer* — an idle Ethernet link is already a timing signal, so sync costs no packets and incurs no queueing uncertainty. HUYGENS changes the *statistics*. Sundial changes the *hardware and topology*.
- **The common failure mode is not the clock, it is the plumbing.** Meta measured ± 100 ns → **-400 μ s** — "over 4000 times difference" — from disabling transparent clocks on a *single* switch.

They All Return an Interval

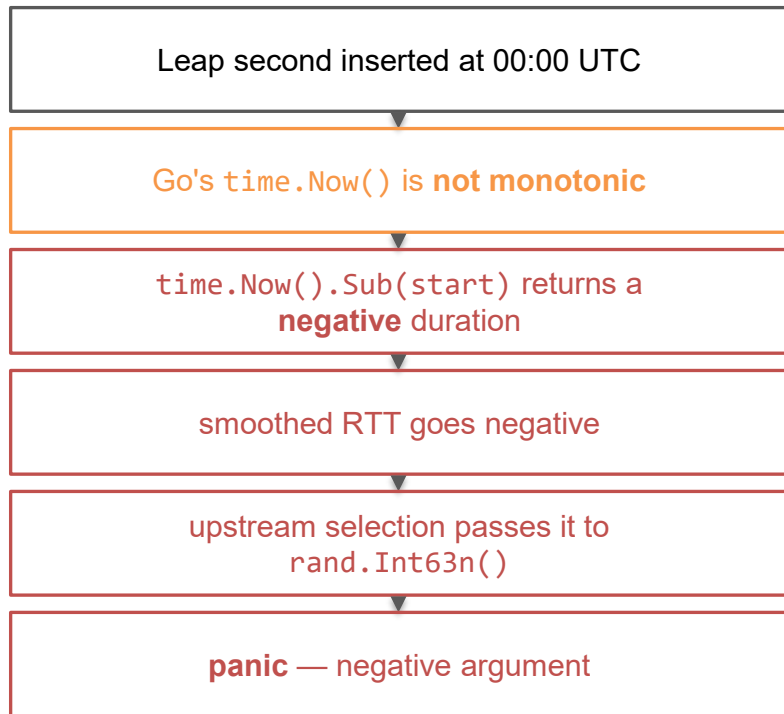
	API	Published bound	Basis
Spanner TrueTime (2012)	<code>TT.now()</code> → [earliest, latest]	ϵ 1–7 ms sawtooth; 4 ms most of the time	GPS + atomic masters, 30 s poll, 200 μ s/s drift
AWS ClockBound (~2021)	<code>now()</code> → (earliest, latest) + status	<100 μs over NTP <40 μs over the PHC	Nitro PHC; error bound read from ENA sysfs
Meta fbclock (2022)	{earliest, latest, window_of_uncertainty}	436 ns at 99.9999% (~92 ns measured)	GNSS 12 + MAC 43 + ts2phc 52 + ptp4l 61 ns

- **A ~10,000× tightening in a decade** — and all three return an *interval*, because a scalar cannot express what they do not know.
- Different **theory of error**, though: Spanner's ϵ is a **worst-case bound**; Meta scales measured variance to a **statistical confidence** (92 ns $\times$ 4.745 $\approx$ 436 ns, six nines).

Leap Seconds End in 2027

- **27 leap seconds**, all positive, since 1972. TAI – UTC = **37 s**. The last one was **31 Dec 2016** — the longest gap of the era.
- **CGPM Resolution 4 (Nov 2022)** said "in, or before, **2035**." **BIPM's CCTF report (July 2026) proposes continuous UTC from 20 May 2027** — eight years earlier.
- **Why the acceleration:** a 2025 BIPM/IERS stochastic model of Earth rotation put a **~30% probability of a *negative* leap second within 10 years**. One has never occurred or been tested; industry told BIPM that preparing for one "could be as demanding as ... the millennium Y2K bug."
- |UT1 – UTC| tolerance becomes **1 hour** ($\approx$ a millennium of headroom). Socially unremarkable: time zones already shift civil time by up to 3 h.
- **Smearing is not standardized** — so smeared clocks disagree with each other *and* with UTC during the window. The CGPM cited this as itself undermining UTC as a unique reference.

Case Study: Cloudflare, 1 Jan 2017



- Impact: **0.2% of DNS queries**, <1% of HTTP, across a subset of machines in 102 data centers. Only **CNAME** customers — they alone require an outbound recursive lookup.
- Resolved 06:45 UTC. **The fix was effectively one character.**
- **The bug was not the leap second.** It was an unstated assumption — *time cannot go backwards* — baked into a language runtime.
- Aftermath: Go 1.9 added a **monotonic clock source**. The lesson outlived the outage.

- Physical clocks
 - Why time is hard
 - Synchronization protocols
 - NTP, PTP, and datacenter time
- **Logical clocks**
 - Lamport clock protocol
- Examples
 - Global log for debugging
 - Mutual exclusion

- Leslie Lamport observed that most coordination in distributed systems — mutual exclusion, barriers, a complete event log — **does not require a global notion of real time**.
- Most of it only needs a **global order of discrete events**.
 - In our debugging example, you only need the order between the *dependent* events that could have caused the failure. The wall-clock time they happened at is incidental.
- And a global **order** is far easier to guarantee than zero-error real-time synchronization — which we just spent 31 slides showing is impossible.
- This is why so many foundational DS protocols rest on logical clocks.

- Lamport stated **two** requirements:
 - **Program order** — the order of events within one process must be preserved by the logical clock.
 - **Message order** — a send event must always precede that message's receive event.
- Together these capture all **internal causality**, by imposing a **happens-before** ($\rightarrow$) relation between any potential cause and any potential effect.
- Formally the guarantee is one-directional:
 - $e \rightarrow f \Rightarrow C(e) < C(f)$
 - The **converse does not hold** — and that asymmetry is the source of both the power and the limitation. We return to it at the end.

Logical Clock Protocol: Setup

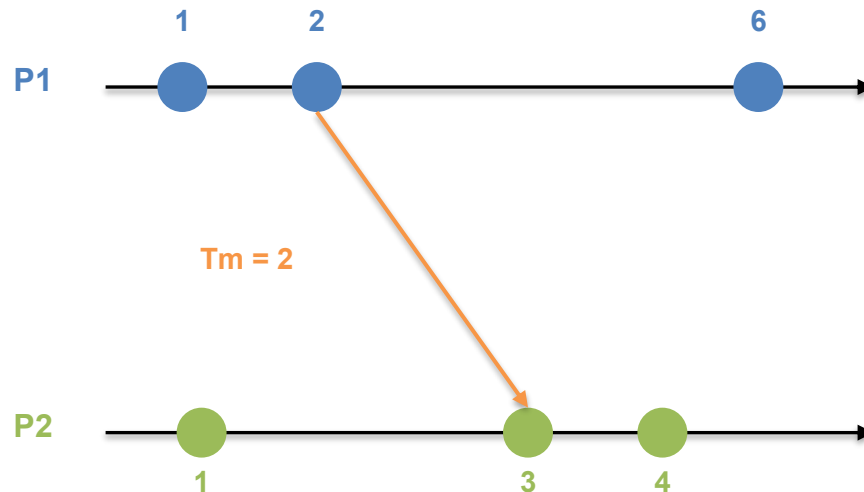
- **Process** — an individual node in the distributed system.
- Processes communicate **only by messages** (e.g. RPCs). No shared memory, no shared clock.
- **Events** may be message sends/receives, or system-specific local events — a file write, a database commit, whatever is meaningful for the system.
- Model each process as a **state machine**: an initial state, and events that move it from one state to the next.
 - This is the same state-machine framing we will use for replication and consensus later in the course. It is worth getting comfortable with now.

Lamport Clock Protocol

Each process P_i keeps a counter C_i .

1. P_i increments C_i between any two successive events.
2. When P_i sends message m , it piggybacks $T_m = C_i$.
3. When P_j receives m it syncs clocks and advances the clock as he receipt is itself an event:

```
[ sync ]   $C_j := \max(C_j, T_m)$   
[ recv ]   $C_j = C_j + 1$ 
```



P_2 receives with $T_m = 2$ while $C_2 = 1$, so $C_2 := \max(1, 2) = 2$, then the receipt advances it to 3.

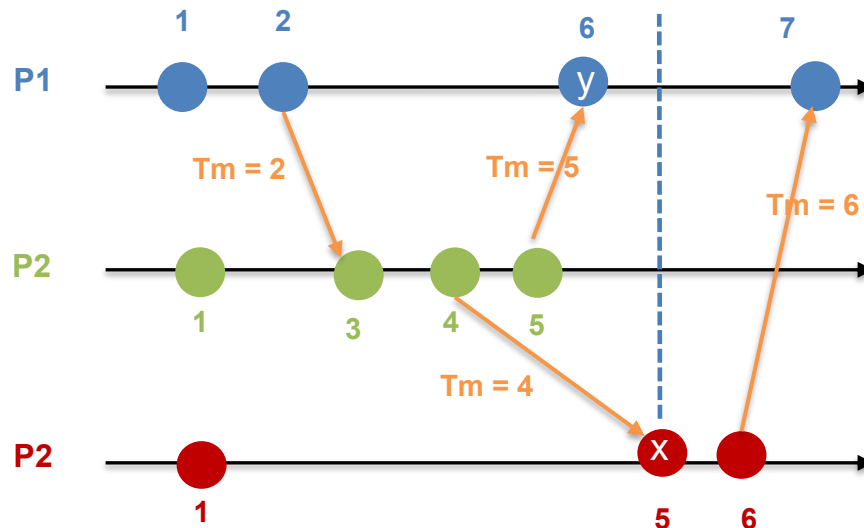
Note C2 skips 2 entirely — a logical clock is a watermark of causal knowledge, not a count of local events.

Lamport Clock Protocol with 3 processes

Each process P_i keeps a counter C_i .

1. P_i increments C_i between any two successive events.
2. When P_i sends message m , it piggybacks $T_m = C_i$.
3. When P_j receives m it syncs clocks and advances the clock as he receipt is itself an event:

[sync]	$C_j := \max(C_j, T_m)$
[recv]	$C_j = C_j + 1$



Formally the guarantee is one-directional:

$$e \rightarrow f \Rightarrow C(e) < C(f)$$

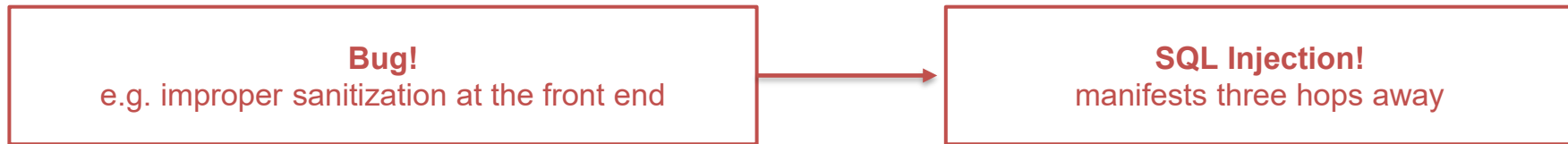
The **converse** does not hold in this example

$$C(x) < C(y) \not\Rightarrow x \rightarrow y$$

From Partial Order to Global Order

- The protocol gives a **partial order** — it says nothing about two events that never causally influenced each other (*concurrent* events).
- Often we need a **total order** that every process agrees on.
- Break ties with the **process ID**: timestamp each event $C_i.i$, and compare lexicographically or consider a tuple $\langle C_i, i \rangle$ and compare C_i and tie breaker i
 - The result is a total order that is **consistent with** causality: if $e \rightarrow f$ then e sorts before f . Concurrent events get *some* order, chosen arbitrarily but consistently.
- That arbitrariness is fine for a debugging log or a lock queue — and, as we will see, is exactly what makes it **too much order** for some systems.

Example: Distributed Debugging (recap)



```
SELECT * FROM Users WHERE id='123'; DELETE * FROM Users
```

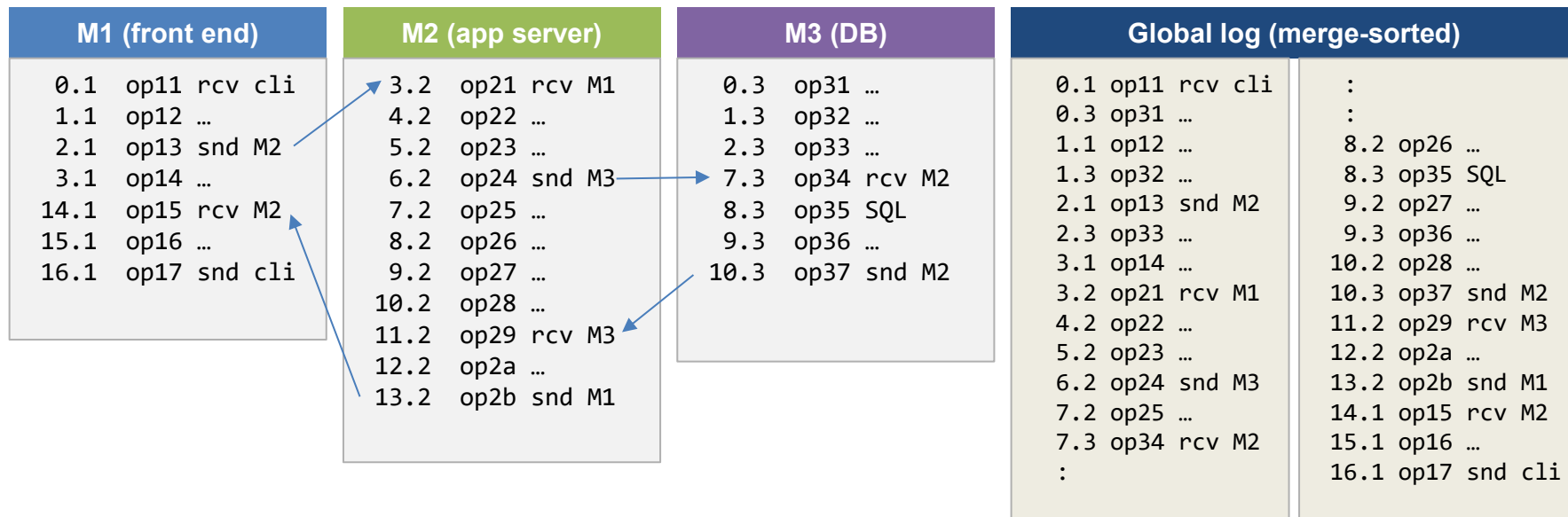
The cause and the symptom are on different machines.

Activity: Build the Global Log

M1 (front end)	M2 (app server)	M3 (DB)	Global log (merge-sorted)
0.1 op11 rcv cli 1.1 op12 ... ?? op13 snd M2 ?? op14 ... ?? op15 rcv M2 ?? op16 ... ?? op17 snd cli	?? op21 rcv M1 ?? op22 ... ?? op23 ... ?? op24 snd M3 ?? op25 ... ?? op26 ... ?? op27 ... ?? op28 ... ?? op29 rcv M3 ?? op2a ... ?? op2b snd M1	0.3 op31 ... ?? op32 ... ?? op33 ... ?? op34 rcv M2 ?? op35 SQL ?? op36 ... ?? op37 snd M2	0.1 op11 rcv cli 0.3 op31 ... ? ? merge-sort the three logs by timestamp

Timestamp every op with $C_{i.i}$ (assume $C_i = 0$ initially), then **merge-sort** the three logs.

Activity: Solution



Note: how **M2's clock jumps** on receiving from M3 — the $\max()$ rule propagating causal knowledge.

Similar: **M1's clock jumps** on receiving from M2

SQL injection point (8.3=op35) now sits AFTER front end operation (2.1=op13)

Example: Mutual Exclusion

- **Goal.** At most one process in the critical section at a time, and requests served in a **fair, agreed order** — no central lock server.
- Why it is a good test of logical clocks: correctness needs only that **every process agrees on who asked first**. The wall-clock instant is irrelevant.
- Lamport's solution: a **shared priority queue**, replicated at every process and ordered by Lamport timestamp.
 - Each process i keeps Q_i , its own replica of the queue.
 - To enter, a process must hold **replies from all other processes AND** be at the **front of its own Q_i** .
- The rest of this section is the algorithm and a worked three-process execution.

Why Those Two Conditions?

Condition 1: replies from everyone

Guarantees every other process is
aware of your request

Condition 2: front of your own Qi

Guarantees you are **aware of every**
earlier request **for the mutex**

Together: everyone knows about you, and you know about everyone earlier. So the front of the queue is the same process at every replica — **without any process ever seeing the global state.**

- Why replies are needed at all: a message from a process with an *earlier* request might still be **in flight**. The reply is proof it has none.
- This is why the algorithm needs **N-1 replies** and cannot be optimized down without weakening the guarantee.

Mutual Exclusion: The Algorithm

To REQUEST the mutex:

- stamp the request $\langle T, i \rangle$ with your Lamport clock
- add $\langle T, i \rangle$ to your own Q_i
- broadcast the request to all other processes

On RECEIVING a request $\langle T, j \rangle$:

- add $\langle T, j \rangle$ to Q_i
- reply to j (includes C_i)

To ENTER the critical section:

- wait until (a) $\langle T, i \rangle$ is at the front of Q_i
and (b) a reply has arrived from every other process

To RELEASE:

- remove $\langle T, i \rangle$ from Q_i
- broadcast the release to all other processes

Queue entries are ordered by $\langle T, i \rangle$ **lexicographically** — the total order from the previous section. That is the only reason all the replicas agree.

Mutual Exclusion: Execution

Process 1	Process 2	Process 3
42 (start)	11 (start)	14 (start)
43 recv <15,3>	16 recv <15,3>	15 request <15,3>
44 reply 1 → <15,3>	17 reply 2 → <15,3>	18 recv reply 2
45 request <45,1>	18 request <18,2>	45 recv reply 1

Queues after the requests propagate: $Q_1 = Q_2 = Q_3 = \langle 15,3 \rangle, \langle 18,2 \rangle, \langle 45,1 \rangle$

Three processes request at local times **15, 18 and 45** — and every replica of the queue ends up in the **same order**, even though no process ever saw the others' clocks directly.

Note process 1's clock reads **45** when it requests, while process 3's read **15**. In real time they may have asked in any order — the algorithm does not care, and does not need to.

Mutual Exclusion: Who Enters?

Process 1

```
42 (start)
43 recv <15,3>
44 reply 1 → <15,3>
45 request <45,1>
49 recv reply 3
50 recv <18,2>
51 reply 1 → <18,2>
```

Process 2

```
11 (start)
16 recv <15,3>
17 reply 2 → <15,3>
18 request <18,2>
51 recv reply 3
52 recv <45,1>
53 recv reply 1
54 reply 2 → <45,1>
```

Process 3

```
14 (start)
15 request <15,3>
18 recv reply 2
45 recv reply 1
46 run crit. sec...
47 recv <45,1>
48 reply 3 → <45,1>
49 recv <18,2>
50 reply 3 → <18,2>
```

Process 3 enters at $t = 46$.

It is at the front of Q_3 and holds replies from both 1 ($t=45$) and 2 ($t=18$).

Process 1 waits — it holds all replies by $t=49$, but is **last** in Q_1 . Both conditions are required, not either.

Lamport Clocks: Pluses, Minuses

- (+) They capture **all internal causality**, by imposing **happens-before** on every pair of events that could influence one another. Enough for a great many coordination problems.
- (-) Internal causality is sometimes **not enough**: events *outside* the system can causally influence it, and Lamport clocks cannot see them.
 - The classic case: a user reads a value on one machine and then acts on another. The causal link ran through the user, not through a message. *We will see this in Spanner.*
- (-) $C(e) < C(f)$ does **not** imply $e \rightarrow f$ — only *potential* causality. So, the order can be **too much order**, serialising work that was genuinely independent and costing concurrency.
- (-) A single scalar cannot tell you *which* events are concurrent — only that some order exists.

Fixing the Converse: Vector Clocks



COMS4113

- Each process keeps a **vector** of N counters, one per process, instead of a single integer [Fidge-1988, Mattern-1988].
- The relation becomes an **if and only if**:
 - $e \rightarrow f \Leftrightarrow \mathbf{VC}(e) < \mathbf{VC}(f)$, where "<" means $\leq$ in every component and $<$ in at least one
 - So **incomparable vectors mean genuinely concurrent events** — a question Lamport clocks cannot answer at all.
- Cost: **$O(N)$** state and $O(N)$ on every message. "The space requirement of VC is on the order of nodes in the system, and is prohibitive."
- For our running example, the payoff is exact: **Lamport clocks find one consistent global log; vector clocks find all of them** — which is what you want when you do not yet know which cut exposes the bug.
- In production: **Dynamo** (SOSP '07) (AWS Highly Available KV store) for conflict detection; version vectors and *dotted* version vectors are the descendants.

- **Shift from Execution to State:** Instead of tracking process message paths, entries represent logical counters attached to **replicated data items** (keys).
- **Detecting Concurrent Conflicts:** Two versions are concurrent if neither vector dominates the other : $(V_1 \not\leq V_2)$ and $(V_2 \not\leq V_1)$
- **Dynamo's Resolution Strategy:** When a conflict is found, the storage node retains both variations as **siblings** and delegates resolution to the application layer (e.g., merging shopping carts).
- **The Cost of High Availability: Sibling Explosion.** Multiple overlapping writes without a strict *Read-Modify-Write* sequence can cause un-reconciled siblings to grow exponentially over time.

- **Decoupled Architecture:** Separates the broad causal history context (the Vector) from the specific event that created a sibling value (the **Dot**).|
- **The "Dot" Identifier:** Each unique version value is tagged with a discrete (Node, Counter) dot representing its exact mutation event.
- **Precise Sibling Pruning:** Allows database nodes to cleanly identify exactly which older siblings are causally dominated by an incoming write.
- **The Payoff:** Obsolete variants are **instantly overwritten and deleted**, preventing structural data bloating in high-throughput production environments.

- "There is a gap between the theory and practice of distributed systems in terms of the use of time" — theory shunned physical time; practice used it best-effort [Kulkarni-2014].
- **HLC** keeps a logical clock that is *always close to* the physical clock, and **fits in a 64-bit NTP timestamp**.
 - Captures causality like a logical clock, *and* is usable in place of a physical clock. It **reads** physical clocks and never updates them, so it runs alongside NTP without interference.
- The contrast with TrueTime is the point:
 - Spanner must **delay** an event to wait out ϵ — "such delays and reduced concurrency are prohibitive especially under looser clock synchronization"
 - HLC returns consistent snapshots "**without needing to wait out clock synchronization uncertainties**" — and needs no special hardware
- In production: **CockroachDB** and **YugabyteDB** (both cite HLC directly); MongoDB's cluster time.

- **Next class:** we step away from Lamport clocks — and then come straight back to them.
- Lamport clocks underpin a great many coordination protocols, including those that solve **consensus** — the central coordination problem of this course, with many instantiations.
- We will formulate consensus and its relatives first, then return to logical clocks when we build the solutions.
- **A 2026 research note.** Some recent work *inverts* today's framing: rather than eliminating clock error, accept it and reason probabilistically.
 - "Beyond Lamport, Towards Probabilistic Fair Ordering" (2025) proposes a **likely-happened-before** relation annotated with a probability — and notes that the relation is **intransitive**.
 - A fitting closing thought: the relation we spent today making crisp may be worth deliberately blurring.

- **[Lamport-1978]** L. Lamport. *Time, Clocks, and the Ordering of Events in a Distributed System*. CACM 21(7):558–565, 1978.
- **[Mills-1991]** D. Mills. *Internet Time Synchronization: The Network Time Protocol*. IEEE Trans. Communications 39(10):1482–1493, 1991.
- **[RFC 5905]** Mills, Martin, Burbank, Kasch. *NTPv4 Specification*. June 2010. — and RFC 8915 (NTS, 2020).
- **[Marzullo-1983]** Marzullo & Owicki. *Maintaining the Time in a Distributed System*. PODC 1983, pp. 295–305.
- **[Fidge-1988] / [Mattern-1988]** Vector clocks (ACSC'88, pp. 56–66; Workshop on Parallel & Distributed Algorithms, pp. 215–226).
- **[Kulkarni-2014]** Kulkarni, Demirbas, Madappa, Avva, Leone. *Logical Physical Clocks*. OPODIS 2014, pp. 17–32.
- **[Corbett-2012]** Corbett et al. *Spanner: Google's Globally-Distributed Database*. OSDI 2012.

Further Reading: Datacenter Time

- **HUYGENS** — Geng, Liu, Yin, Naik, Prabhakar, Rosenblum, Vahdat. NSDI 2018, pp. 81–94. Coded probes + SVM; 12–15 ns mean at 40% load.
- **DTP** — Lee, Wang, Shrivastav, Weatherspoon. *Globally Synchronized Time via Datacenter Networks*. SIGCOMM 2016, pp. 454–467.
- **Sundial** — Li et al. OSDI 2020, pp. 1171–1186. ~100 ns bound across failure types; cuts Spanner commit-wait 211 → 49 μ s (median).
- **Graham** — Najafi & Wei. NSDI 2022, pp. 453–466 (Best Paper). Drift 200 ppm → 100 ppb.
- **Firefly** — Namyar et al. *Scalable, Ultra-Accurate Clock Synchronization for Datacenters*. SIGCOMM 2025, pp. 434–452. Sub-10 ns.
- **IEEE 1588-2019** (PTP v2.1) and **1588g-2022**; ITU-T **G.810** for the jitter/wander/drift vocabulary.
- **BIPM CCTF**, *Report on Draft Resolution C*, July 2026 — continuous UTC.